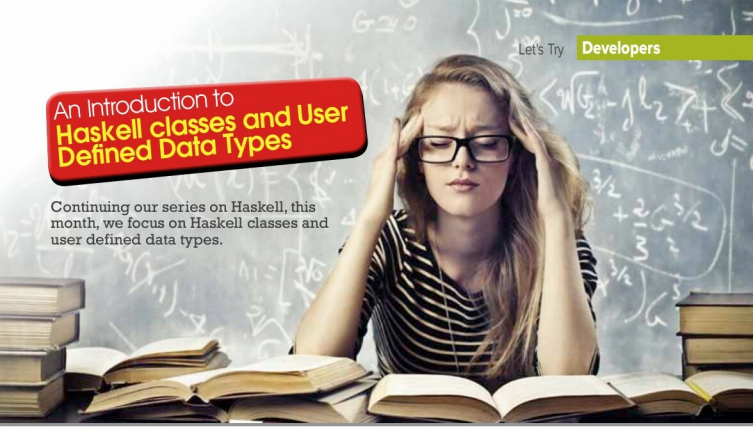
An Introduction to Haskell classes and User Defined Data Types



Continuing our series on Haskell, this month, we focus on Haskell classes and user defined data types.

Haskell is a purely functional programming language and it enforces strictness with the use of types. In this article, we shall explore type classes and user defined data types.

Consider the *elem* function that takes an element of a type, a list, and returns 'true' if the element is a member of the list; and if not, it returns 'false'. For example:

```
ghci> 2 `elem` [1, 2, 3]
True
ghci> 5 `elem` [1, 2, 3]
False
```

It's type signature is shown below:

```
ghci> :t elem
elem :: Eq a => a -> [a] -> Bool
```

The type signature states that the type variable 'a' must be an instance of class 'Eq'. The class constraint is specified after the '::' symbol and before the '=>' symbol in the type signature. The *elem* function will thus work for all types that are instances of the *Eq* class.

The word 'class' has a different meaning in functional programming. A type class is a parametrised interface that defines functions. A type that is an instance of a type class needs to implement the defined functions of the type class. The *Eq* class defines functions to assert if two type values are equal or not. Its definition in Haskell is as follows:

```
class Eq a where
    (==), (/=)              :: a -> a -> Bool
    x /= y                  = not (x == y)
    x == y                  = not (x /= y)
```

The keyword *class* is used to define a type class. This is followed by the name of the class (starting with a capital letter). A type variable ('a' here) is written after the class name. Two functions are listed in this class for finding if two values of a type are equal or not. A minimal definition for the two functions is also provided. This code is available in *libraries/ghc-prim/GHC/Classes.hs* in the ghc (Glasgow Haskell Compiler) source code.

The example works for integers '2' and '5' in the example above, because they are of type *Int*, which is an instance of *Eq*. Its corresponding definition is given below:

```
Instance Eq Int where
    (==) = eqInt
    (/=) = neInt
```

The keyword *instance* is used in the definition followed by the name of the class *Eq*, and a specific type *Int*. It uses two primitive functions *eqInt* and *neInt* for checking if the given integers are equal or not. The detailed definition is available in *libraries/ghc-prim/GHC/Classes.hs* in the ghc source code.

There are a number of pre-defined type classes available in the Haskell platform.

The *Ord* type class denotes types that can be compared. The 'compare' function will need to be implemented by types that want to be instances of this class. The resultant values of 'compare' are GT, LT, or EQ. For example:

```
ghci> 'p' > 'q'
```